Reach: I just wanted the character to walk

Building with Claude while keeping sight of the game.

Let Claude do more of the implementation while you stay involved in the decisions.

One adventurer. One warm fireplace. One door that complicates everything.



A story about intent, implementation, and knowing what is true.

The paperwork got there first

More instructions. More checks. Still a character who needed to walk.

01 1,077 decision records and 662,000 lines of documentation.

02 An 8,275-line verification gate that still missed defects.

03 The active milestone: "click a character and walk them somewhere."

04 New instructions kept arriving. Old ones rarely left.



The process grew. The playable goal stayed small.

Keep the current rule. Let Git remember the old one.

Meet Pip. Pip would like to be warm.

A fictional game, a cozy cottage, and a fireplace just out of reach.

01 Pip is outside. The fireplace is inside.

02 The player clicks a spot beside the fireplace.

03 In our first version, the door is the only usable entrance.

04 "Just add click-to-move." Famous last words.



One destination: the warm fireplace inside.

The tests pass. Pip has questions.

"It works. It just isn't what I asked for."

01 With the door open, Pip reaches the fireplace.

02 With it closed, should Pip wait, refuse, or go somewhere else?

03 Claude can implement any of those choices.

04 We still need to decide which one belongs in this game.



The route is blocked. The design decision is still open.

Catch an unanswered design question before it becomes an accidental feature.

Give the design conversation its own place

Reach is a Claude Code plugin built around three kinds of work.

IDEATE

/reach:ideate: decide what we are building and record the contract.

Your judgment

BUILD

/reach:build: implement and prove the layers outside presentation.

Automated evidence

MILESTONE

/reach:milestone: build and test presentation; hand you the experience.

Tests + your judgment

Focus your attention on the decisions that need you.

Ideate keeps sight of the forest

/reach:ideate challenges the idea before it becomes a requirement.

01 Ideate raises the tradeoff: a nearby destination could surprise the player.

02 Together, we agree: Pip follows a valid route to the spot the player clicked.

03 If no route exists, Pip stays put and the game explains why.

04 Ideate writes the contract that build will work against.



Decide what the player should expect.

Get useful pushback while the idea is still cheap to change.

Build turns the agreement into working behavior

/reach:build must test the agreed behavior before calling it built.

01 Build writes evidence before or with the code: a closed door leaves Pip in place.

02 It tests Pip’s position and the blocked-route state the presentation reads.

03 It runs the project’s verification tiers, including the actual game build.

04 The verification runner reports unavailable tiers as SKIPPED, never passed.

01 AGREED BEHAVIOR

Closed door → Pip stays put

02 NAMED EVIDENCE

Position + blocked-route state

03 VERIFICATION

Tests → game build → explicit result

Know what was checked, and what still wasn't.

Then someone closes the door

/reach:build records the missing decision and keeps independent work moving.

01 Build finds that the route can disappear after movement starts.

02 It records: "Open: What should Pip do if the door closes on the way?"

03 The question and its evidence go beside the claim for ideate to resolve.

04 Build picks the next independent task instead of inventing the answer.



A blocked claim becomes a question for ideate.

Fewer "continue" prompts. Unanswered design decisions stay yours.

Decide once. Put the answer where it lasts.

/reach:ideate turns the answer into the contract future sessions read.

01 In ideate, we agree: if the route becomes blocked, Pip stops before the obstacle.

02 Pip stays at the last valid position, and the game explains the interruption.

03 Ideate rewrites the claim and publishes the decision.

04 Build syncs the answer, removes its lane's question, and implements the rule.

BUILD → OPEN QUESTION

What if the door closes mid-walk?

↓

IDEATE → UPDATED CONTRACT

Stop before the obstacle.
Keep the last valid position.

Build syncs the answer and removes its Open.

The answer becomes part of the project, not another message to dig up.

The door is locked. What about the window?

/reach:ideate reviews the design beyond the immediate feature.

01 The next request: let Pip reach the fireplace through a window.

02 The easy move is to add window code beside the door code.

03 Now both need rules for opening, closing, locking, and movement.

04 Ideate asks: are these separate systems, or instances of the same idea?



Same destination. A second kind of opening.

Step back before every new feature becomes another special case.

Doors and windows share an opening contract

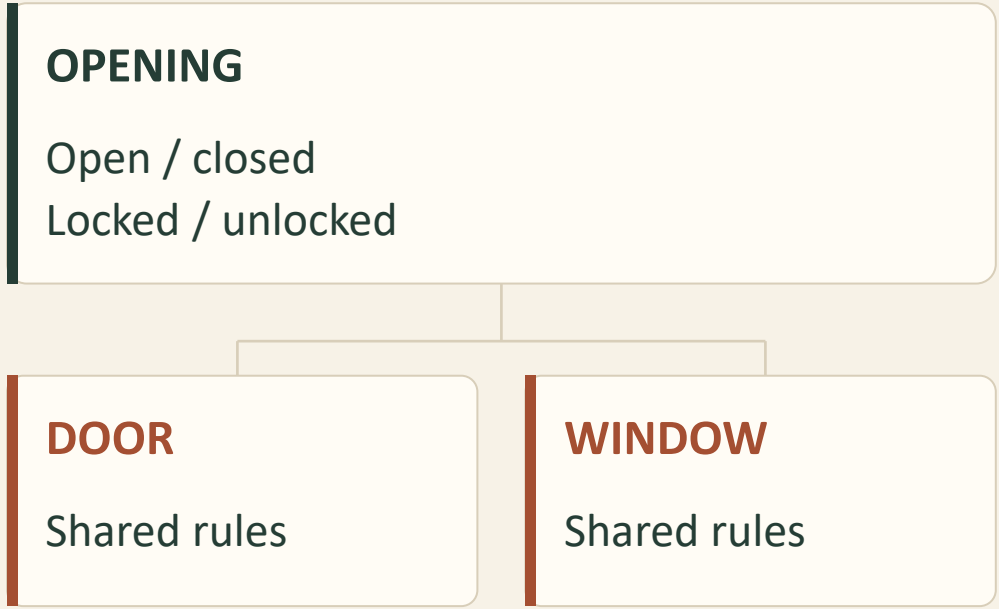
Ideate defines the shared contract. Build implements it.

01 Ideate identifies the common concept: an opening can be open or closed, locked or unlocked.

02 We agree: a lock prevents opening it. Unlocking does not open it.

03 Pip can pass through if it is open, reachable, and large enough.

04 Ideate records those rules; build implements and tests them for doors and windows.



Traverse when open, reachable, and large enough.

Extend the shared model instead of copying the logic.

Catch the bug now. Catch it again later.

/reach:build must show that the test can catch the broken rule.

01 Build tests each opening as the only entrance: when it is closed, Pip must stay put.

02 It deliberately allows passage through a closed opening and requires the test to fail.

03 It restores the correct behavior and requires both cases to pass again.

04 Future build runs rerun those tests so a window change does not break the door.

BREAK IT	Closed opening allows passage
EXPECT RED	The movement assertion fails
RESTORE	Door + window cases pass
KEEP RUNNING	Catch the rule breaking again

Break it once to check the test. Keep the test to catch regressions.

Keep the work separate. Share the decisions.

The skills follow the workflow; Reach's scripts enforce the landing checks.

01 Build and milestone each work in their own Git worktree.

02 Ideate syncs before reading the current contracts.

03 The skills pass the verified commit; the landing script refuses a different merged tree.

04 The publish script shares integration first; a stale lane backup cannot hold it up.

BUILD

Own worktree

MILESTONE

Own worktree

Verified commit → checked merge

INTEGRATION

Publish the accepted decisions and code.

Ideate syncs before the next conversation.

Check the version you actually accept.

Verification takes longer. Keep the work moving.

/reach:iterate carries build work across runs while you follow progress.

01 As Pip's game grows, each full test and build cycle takes longer.

02 /reach:iterate starts a supervisor that runs build in fresh agent processes.

03 Each run finishes its checks; the next picks up from committed work.

04 You can step away, follow progress, or request a stop after the current run.

ONE RUN

Implement → verify → land

↓

FRESH RUN

Read the documents. Pick up the work.

Your console follows progress and relays a stop request.

Explicit opt-in; agent permission prompts disabled.

Long verification no longer means waiting around to start the next run.

Pip can get inside. Does it feel right?

/reach:milestone builds the experience. You decide whether it works.

01 Milestone builds and tests the cues over the movement state build has proved.

02 It drives the walkthrough on the real build, then hands that build to you.

03 You judge: does Pip’s response make sense, or do the controls seem broken?

04 Milestone records your verdict; only your confirmation accepts the experience.



Working behavior still needs a human verdict.

You decide when the experience works.

The tests passed. You found a bug.

The rule was clear. The tests missed a case you encountered while playing.

01 Pip should stop before the door. In your playthrough, he crosses it.

02 You report the bug. Ideate connects it to the movement rule.

03 Build adds a test that catches the crossing, fixes it, and reruns the checks.

04 You retry the real build and confirm the fix.

OWNER FINDS BUG

Pip crossed a closed door.

IDEATE FINDS RULE

Stop before the obstacle.

BUILD TESTS + FIXES

New case catches the crossing.

OWNER RETRIES

Confirm on the real build.

Your feedback becomes a tested repair, and you confirm the result.

Pip gets warm. Tomorrow's session can pick up.

Each skill leaves the next session something concrete to work from.

01 Ideate maintains the current contract: openings, not the old door-only rule.

02 Build records the evidence and leaves unfinished claims and questions visible.

03 Milestone records your acceptance; ideate retires the completed walkthrough.

04 The next build session rereads the documents and picks up the next buildable task.



The warmth is the outcome. The contract carries the understanding.

Less reconstruction when you start again.

What changed in my own work

Pip is fictional. These benefits come from my experience.

01 Ideate gives Claude and me a place to think about the whole system.

02 Keeping the design discussion there makes the interaction feel more human.

03 Separating build’s implementation from ideate’s broader review has improved my code.

04 Ideate helps find shared rules before new features become parallel code paths.



Think together about the whole game.

Where Reach earns its keep

Enough structure to stay in control, with real upkeep attached.

01 Best fit: one owner, a growing project, and more agent output than they can comfortably review.

02 Team repositories can use the verification layer, or run it all over your own tasks.

03 Contracts and worktrees take upkeep. A small utility may not need them.

04 110 controls; used daily on one downstream project. A second adoption is still untested.

Appendix: the practical benefits

What I've observed, and what the design should help with.

01 Ideate: useful pushback, shared rules, and decisions that survive the conversation.

02 Build: fewer "continue" prompts, visible blockers, and evidence against the agreed behavior.

03 Build + ideate: focused implementation followed by broader review in a fresh session.

04 Milestone: a real build to judge, with acceptance kept in your hands.

05 Documents + scripts: easier handoffs, clear verification results, and checked landings.

Appendix: getting started

Install once, then adopt a repository.

01 Requires Claude Code and PowerShell. Objects-mode landing needs Git 2.38 or newer.

02 claude plugin marketplace add SpoiledInk13/Reach

03 claude plugin install reach@reach

04 In Claude Code: /reach:adopt

05 Then use /reach:ideate, /reach:build, and /reach:milestone.

Appendix: where the workflow lives

The decisions and checks live with the repository.

01 process.json: document locations, size limits, evidence conventions, checks, and lanes.

02 Architecture + separately capped roster: shared rules and the list of units.

03 Unit documents: contract readiness, claims, dependencies, and evidence.

04 Walkthroughs: human acceptance, with specific evidence rows as prerequisites.

05 Scripts/reach.ps1: the entry point for verification and lane operations.

Appendix: checking the process

Use the same entry point locally and in automation.

01 pwsh Scripts/reach.ps1 gate — check repository constraints and evidence links.

02 pwsh Scripts/reach.ps1 all — run the gate and automated verification tiers.

03 pwsh Scripts/reach.ps1 prove — exercise the 110 supplied controls.

04 pwsh Scripts/reach.ps1 audit — find adoption gaps and version mismatches.

05 Add custom checks when a real defect shows the need, then prove they can fail.

Appendix: many builders, one game harness

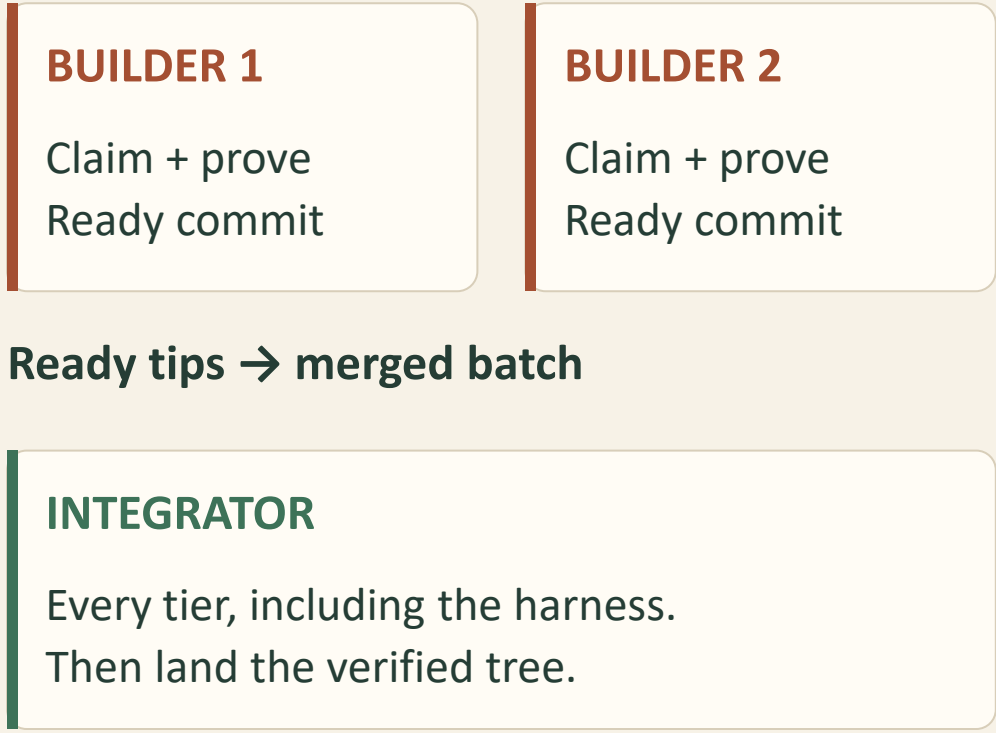
Use the split when an editor, emulator, or device limits parallel work.

01 Each builder claims a unit and proves the tiers that can run without the harness.

02 A builder marks its proven commit ready. It never lands.

03 The integrator merges ready commits, runs every tier over the batch, and lands.

04 When every tier runs anywhere, an ordinary second lane is simpler.



Appendix: your own tasks on a team repository

A personal overlay runs the whole process where the team owns the workflow.

01 One folder git never sees holds process.json, the documents, and the shim.

02 Evidence marks go in the team's tests. Only code and tests reach the team.

03 promote runs every tier on the exact commit, then pushes it for review.

04 Hooks refuse commits off your branches and pushes promote did not record.

05 backup mirrors the folder and commits the mirror, so deletions keep a history.

